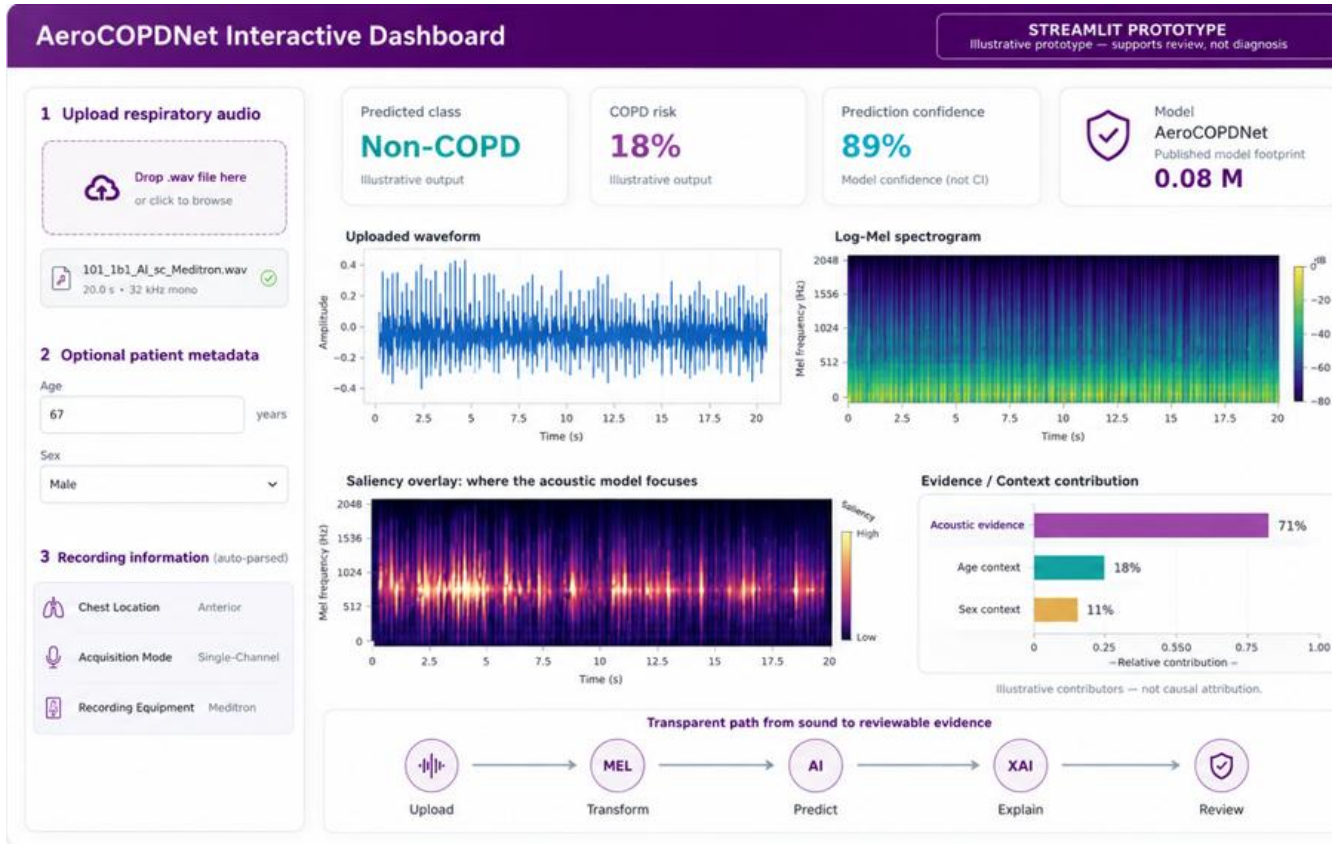


AeroCOPDNet

Interactive Dashboard Tutorial

How to set up, run, use, and interpret the Streamlit interface



Target AeroCOPDNet dashboard interface

Important

This interface is a research/review prototype. It should not be presented as a clinical diagnostic device or used as the sole basis for medical decisions.

1. Quick Start

The dashboard uses your existing PyTorch model code, feature configuration, checkpoint, and a Streamlit front end.

Required project files

```
your_project/
├── dashboard.py
├── config.py
├── model.py
├── checkpoints/
│   └── AeroCOPDNet_fold0.pth
└── <tabular feature JSON referenced by config.TABULAR_FEATURES_JSON>
```

Start the app

1. Open a terminal in the project folder.
2. Activate the Python environment you used for the model, so torch and torchaudio versions remain compatible.
3. Make sure Streamlit, NumPy, and Matplotlib are installed.

```
pip install streamlit numpy matplotlib
```

4. Run the dashboard.

```
streamlit run dashboard.py
```

Streamlit will normally open the app in your browser. If it does not, copy the Local URL shown in the terminal and open it manually.

Recommended

Do not create a completely different Python environment just for the dashboard if your trained model already runs correctly. Reusing the model environment reduces PyTorch/Torchaudio compatibility problems.

2. How the Dashboard Connects to Your Model

The visible dashboard is a front end. The prediction still comes from your trained model and its checkpoint.

Dashboard setting	What it controls
BACKEND_MODEL = "AeroCOPDNet"	Uses AeroCOPDNet_Quad from model.py by default.
AeroCOPDNet_fold0.pth	Checkpoint loaded from config.CHECKPOINTS_DIR.
config.TABULAR_FEATURES_JSON	Defines the order of tabular features expected by the model.
config.SAMPLE_RATE	Target audio sample rate.
config.NUM_SAMPLES	Fixed number of waveform samples supplied to the model.
config.N_FFT / HOP_LENGTH / N_MELS	Parameters used to build the Log-Mel spectrogram.
DISPLAY_MODEL_NAME	Visible model name. Default: AeroCOPDNet.

Dashboard setting	What it controls
DISPLAY_MODEL_FOOTPRINT	Visible published footprint. Model parameter is fixed to 0.08 M.

3. Using the Dashboard

Step 1 - Upload respiratory audio

Use the left-side panel labelled “1 Upload respiratory audio”. Drag a .wav file onto the upload area or click to browse. The dashboard accepts WAV audio only.

- The uploaded signal is converted to mono if it has more than one channel.
- It is resampled to config.SAMPLE_RATE when necessary.
- It is padded or cropped to config.NUM_SAMPLES.
- A Log-Mel spectrogram is produced and passed to the model.

After processing, the file card shows the file name, duration, sample rate, and a green check mark.

Step 2 - Enter optional patient metadata

The “2 Optional patient metadata” area currently exposes Age and Sex. Enter the patient age and choose Male or Female.

Important for your current code

The interface currently supplies Age and Sex only. Any other tabular features listed in the feature JSON are set to 0 unless you explicitly add controls and mapping logic for them.

Step 3 - Review recording information

The “3 Recording information (auto-parsed)” panel displays Chest Location, Acquisition Mode, and Recording Equipment. In the current prototype these visible values are fixed display fields: Anterior, Single-Channel, and Meditron.

Step 4 - Wait for automatic inference

There is no separate Run button. Once a valid audio file is available and the model checkpoint loads successfully, the dashboard automatically runs inference and updates the result cards and plots.

4. Understanding the Result Cards

Card	Meaning
Predicted class	Shows COPD when the COPD probability is at or above the configured threshold; otherwise Non-COPD.
COPD risk	Displays the model probability assigned to COPD as a percentage.
Prediction confidence	Displays $\max(p, 1-p)$. This is model confidence, not a statistical confidence interval.
Model	Always displays AeroCOPDNet.
Published model footprint	Always displays 0.08 M in the current dashboard.

Classification threshold

The default threshold is 0.50. For example, a COPD probability of 0.62 is displayed as COPD; a probability of 0.18 is displayed as Non-COPD.

5. Understanding the Visual Evidence

Uploaded waveform

This chart shows audio amplitude over time. It is useful for checking whether the file contains a visible respiratory signal, silence, clipping, or strong transient noise.

Log-Mel spectrogram

This is the time-frequency representation generated from the uploaded waveform. The horizontal axis represents time, the vertical axis represents Mel-frequency range, and the color intensity represents spectral energy in decibels.

Saliency overlay: where the acoustic model focuses

The dashboard calculates a lightweight gradient-by-input saliency map when possible. Bright or high-saliency regions indicate parts of the spectrogram that had stronger local influence on the output.

Interpretation limit

Saliency is an explanatory aid, not proof that a highlighted respiratory sound caused the prediction. It should be treated as review support only.

Evidence / Context contribution

The bar chart summarizes relative contribution scores for Acoustic evidence, Age context, and Sex context. The current implementation normalizes gradient-by-input magnitudes into relative shares.

- Purple: Acoustic evidence
- Teal: Age context
- Orange: Sex context

If gradient attribution cannot be computed, the code falls back to illustrative values. Therefore, always keep the dashboard note “Illustrative contributors - not causal attribution.”

6. Data Flow Inside dashboard.py

Stage	Function / component	What happens
1. Upload	process_audio()	Loads WAV, converts to mono, resamples, pads/crops, creates Log-Mel spectrogram.
2. Metadata	process_tabular()	Builds the feature vector from Age and Sex using the feature JSON order.
3. Predict	run_inference()	Calls FusionNet_Quad and converts logits to a COPD probability.
4. Explain	explain_with_input_gradients()	Creates saliency and relative audio/age/sex contribution

Stage	Function / component	What happens
		scores.

The bottom visual pipeline summarizes the same process as: Upload -> MEL -> AI -> XAI -> Review.

7. Troubleshooting

Problem	What to check
Feature list missing	Check that config.TABULAR_FEATURES_JSON points to an existing JSON file. If your workflow generates this file during preprocessing, run that preprocessing first.
Weights not found	Check config.CHECKPOINTS_DIR and confirm FusionNet_fold0.pth exists in that folder.
Model loading failed	Make sure model.py matches the architecture used to create the checkpoint. Also check whether the checkpoint stores state dict inside a wrapper.
Audio processing failed	Confirm the file is a readable WAV file. If the problem is sample-rate related, verify your installed torchaudio version is compatible with torch.
Dashboard is blank or old after editing	Stop Streamlit with Ctrl+C and run streamlit run dashboard.py again. A hard browser refresh can also help.
Evidence chart looks illustrative	If gradients cannot be calculated, the dashboard uses fallback contribution values. Check the model forward pass and whether gradients are available for both inputs.
Wrong model is being used	Verify BACKEND_MODEL. The current intended default is AeroCOPDNet, which maps to FusionNet_Quad.

8. Suggested Demonstration Workflow

1. Open the dashboard and point out the purple AeroCOPDNet header and the “STREAMLIT PROTOTYPE” label.
2. Upload one respiratory WAV recording in the left panel.
3. Confirm the green check mark and show the parsed file duration/sample rate.
4. Enter Age and Sex.
5. Explain that inference updates automatically.
6. Read the Predicted class, COPD risk, and Prediction confidence cards.
7. Show the waveform and Log-Mel spectrogram as the signal and transformed representation.
8. Show the saliency overlay as an XAI review aid.
9. Explain the Evidence / Context contribution chart and clearly state that it is not causal attribution.
10. Finish with the Upload -> MEL -> AI -> XAI -> Review pipeline.

Appendix - Essential Commands

```
# Go to your project folder
cd path\to\your_project

# Activate your existing model environment
# Example on Windows:
.venv\Scripts\activate

# Install front-end dependencies if needed
pip install streamlit numpy matplotlib

# Start the dashboard
streamlit run dashboard.py
```